

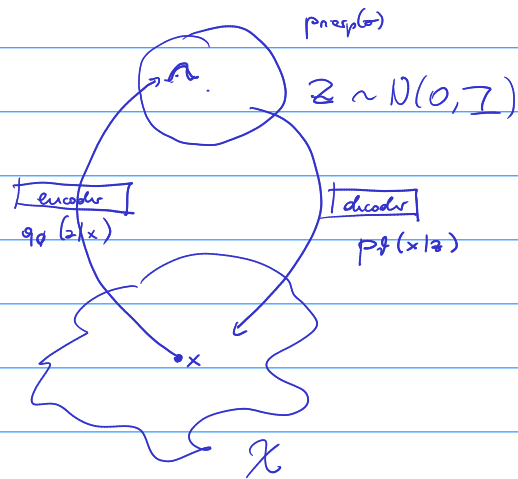
12.3 Variational Autoencoders

Kipf & Welling 2013

Notation: decoder $p_\theta(x|z)$ encoder $q_\phi(z|x)$

Instead of mapping x to a fixed vector in latent space, VAEs map x to a distribution in latent space.

- prior $p_z(z)$ on a latent encoding vector z
- likelihood $p_\theta(x|z)$ of the data given z
- posterior $p_\theta(z|x)$ which will be approximated by $q_\phi(z|x)$



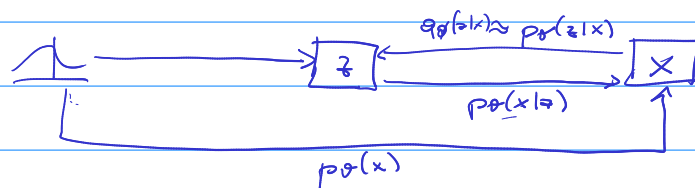
With the usual approach for generative models, we need to maximize:

$$\mathcal{J}^* = \arg \max_{\theta} \prod_{n=1}^N p_\theta(x^{(n)})$$

$x_1, \dots, x_N$

In principle, we would get an approximation $p_\theta(x) = \int_{z \in \mathcal{Z}} p(x|z) \cdot p(z) dz$

To do this effectively we introduce $q_\phi(z|x)$ that approximates $p_\theta(z|x)$



"Interlude on Divergences"

$$D(\cdot \parallel \cdot) : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$$

$\mathcal{S} \dots$ space of prob. distributions

$$\cdot D(p \parallel q) \geq 0 \quad \forall p, q \in \mathcal{S}$$

$$\cdot D(p \parallel q) = 0 \Leftrightarrow p = q$$

Kullback-Leibler Divergence (KL):

$$\text{for discrete distributions} \quad D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) \cdot \log \frac{P(x)}{Q(x)}$$

$$\text{continuous distributions} \quad D_{KL}(p \parallel q) = \int_{x \in X} p(x) \cdot \log \frac{p(x)}{q(x)} dx$$

Remark: Thus KL-Div can also be viewed as an expectation: $D_{KL}(p \parallel q) = E_{x \sim p(x)} \left[\log \frac{p(x)}{q(x)} \right]$

12.3.2.2 ELBO and loss function

ELBO is a lower bound on the likelihood of the data. Introduce KL divergence as a kind of "distance" between $p_\theta(z|x)$ and $q_\phi(z|x)$.

! Derivation of the lower bound!

$$\begin{aligned} \underbrace{\log p_{\theta}(x)}_{\text{likelihood}} &= E_{q(z|x)} [\log p_{\theta}(x)] & \begin{cases} p(z,x) = p(z|x) \cdot p(x) \\ \Leftrightarrow p(x) = \frac{p(x,z)}{p(z|x)} \end{cases} \\ &= E_{q(z|x)} \left[\log \left(\frac{p(z,x)}{p(z|x)} \right) \right] \\ &= E_{q(z|x)} \left[\log \left(\frac{p(x,z)}{q(z|x)} \cdot \frac{q(z|x)}{p(z|x)} \right) \right] = \\ &= E_{q(z|x)} \left[\log \left(\frac{p(x,z)}{q(z|x)} \right) \right] + E_{q(z|x)} \left[\log \left(\frac{q(z|x)}{p(z|x)} \right) \right] \\ &\quad \underbrace{\hspace{10em}}_{\text{ELBO}} \quad \underbrace{\hspace{10em}}_{D_{KL}(q(z|x) \parallel p(z|x)) \geq 0} \\ &\geq \underbrace{E_{q(z|x)} \left[\log \left(\frac{p(x,z)}{q(z|x)} \right) \right]}_{\text{ELBO}} \end{aligned}$$

Our derivation expresses the likelihood as a sum of ELBO and the KL divergence between $q(z|x)$, the posterior estimate, and $p(z|x)$, the true posterior.

The ELBO can be expressed as $\mathcal{L}(x) = \mathbb{E}_q [\log(p_\theta(z) - \log q(z|x))]$

it is also a lower bound on the likelihood:

$$\mathcal{L}(x) = \log p_\theta(x) - \underbrace{D_{KL}(q_\theta(z|x) \| p_\theta(z))}_{\geq 0}$$

$$\Rightarrow \mathcal{L}_{\theta,0}(x) \leq \log p_\theta(x)$$

The KL divergence can be interpreted in two ways:

- a) KL div between the true and the approximate posterior
- b) Gap between ELBO and likelihood.

Next aim: Restructure the KL divergence term to make it tractable with DL approaches.

$$D_{KL}(q(z|x) \| p(z|x)) = \mathbb{E}_{q(z|x)} \left[\log \left(\frac{q(z|x)}{p(z|x)} \right) \right] \\ = \mathbb{E}_{q(z|x)} \left[\log \frac{q(z|x) \cdot p(x)}{p(z,x)} \right] =$$

$$= \log p(x) + \mathbb{E}_{q(z|x)} \left[\log \frac{q(z|x)}{p(x|z) \cdot p(z)} \right] =$$

$$= \underbrace{\log p(x)}_{?1} + \underbrace{\mathbb{E}_{q(z|x)} \left[\log \frac{q(z|x)}{p(z)} \right]}_{D_{KL}(q(z|x) \| p(z))} - \underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{? squared loss?}}$$

Combining ^{Eq} (1) and Eq (2), we obtain:

"VAC loss"

$$\mathcal{L}_{\theta,\phi}(x) = \underbrace{\mathbb{E}_{q(z|x)} \log p_\theta(x|z)}_{\text{decoder loss}} - \underbrace{D_{KL}(q_\phi(z|x) \| p_\theta(z))}_{\text{encoder loss}}$$

This is the objective that the VAEs will maximize/minimize.

The expectation will require to sample $z \sim q_\phi(z|x)$. Sampling is a statistical process, and usually does not allow for gradients to be backpropagated over.

- Solution for the first term: 12.3.3 "Reparameterization trick"

- Solution for the second term: 12.3.5 "KL div between two gaussian variables"

GOAL: Learn θ and ϕ by backpropagation.

12.3.3 Reparametrization Trick

This trick allows to express our random variable z as a deterministic variable with random components: $z = T(\underline{x}, \underline{\epsilon})$. The main idea is to separate the variable into a deterministic and a probabilistic part.

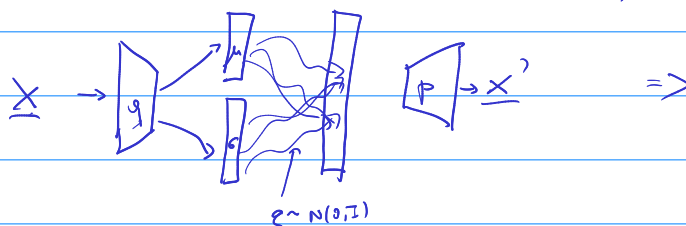
Example: MV Gaussian

$$z \sim q(z|x) = N(z; \mu, \sigma^2 I)$$

$$z = \underbrace{\mu}_{\text{det}} + \underbrace{\sigma}_{\text{rand}} \epsilon, \text{ where } \epsilon \sim N(0, I) \quad \text{and } d$$

$$\frac{\partial L}{\partial \mu} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial \mu} \quad \text{and for } \epsilon \text{ similarly,}$$

Solution: our neural network $q_\theta(z|x)$ ^{outputs} predicts a mean μ and a σ vector:



12.3.5 Closed form of the VAE objective: KL-div. of Gaussian RV.

What is still missing is: $D_{KL}(q_\theta(z|x) || p(z))$

- prior $p(z)$ is Gaussian with mean μ_p and diagonal covariance $N(\mu_p, \sigma_p^2 I)$
- $q_\theta(z, x)$: Gaussian with predicted mean μ_q and ^{var.} ~~var.~~ $\sigma_q^2 I$

There is a closed form solution, i.e., an expression, for the KL-div between Gaussians:

$$D_{KL}(q_\theta(z|x) || p(z)) = \log\left(\frac{\sigma_p}{\sigma_q}\right) + \frac{(\mu_q - \mu_p)^2 + \sigma_q^2}{2\sigma_p^2} - \frac{1}{2}; \quad \text{we want } \left[\begin{matrix} \mu_p = 0 \\ \sigma_p = 1 \end{matrix} \right]$$

$$D_{KL} = -\log(\sigma_q) + \frac{1}{2} \mu_q^2 + \frac{1}{2} \sigma_q^2$$

$$-D_{KL}(q(z|x) || p(z)) = \frac{1}{2} (1 + \log(\sigma_q^2) - \sigma_q^2 - \mu_q^2) \quad \begin{matrix} \text{Loss?} \\ \text{Diff?} \end{matrix}$$

zero, iff $\sigma_q = 1$ and $\mu_q = 0$; Too: check!

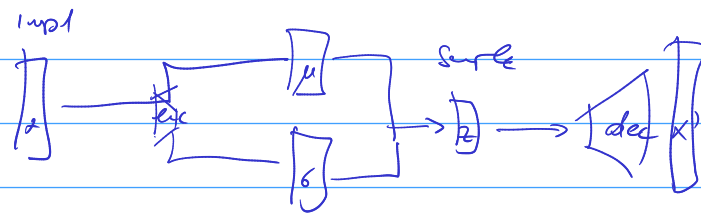
If the posterior $q(z|x)$ matches the prior with $\mu_z = 0$ and $\sigma_z = 1$, then the KL-div is zero.

We now investigate the "VAE loss":

$$\begin{aligned} \mathcal{L}_{\phi, \psi}(x) &= \mathbb{E}_{q(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \| p_{\theta}(z)) \\ &= \mathbb{E}_{q(z|x)} [\log p_{\theta}(x|z)] - \sum_{j=1}^J \frac{1}{2} (1 + \log(\sigma_j^2) - \sigma_j^2 - \mu_j^2) \end{aligned}$$

where $\underline{\sigma} = (\sigma_1, \dots, \sigma_J)$ and $\underline{\mu} = (\mu_1, \dots, \mu_J)$ are the mean and std dev predicted by the encoder network. Note that in a practical implementation the network usually predicts $\log \sigma_j$ and not σ_j directly.

12.3.4 Putting everything together



$$\mathcal{L}(x, \hat{x})$$

↓ ~~KL~~ $\log p_{\phi}(x|z)$

Pseudo-Code

```
# provide: encoder(), decoder(), with encoder_weights, decoder_weights, also x
while TRAINING:
    x = sample_batch(X)
    z_mean, z_varz_var = encoder(x)
    noise = sample_noise() # from 0,1
    z = z_mean + noise *  $\sqrt{z\_var}$  # we have sampled from Z space
    output = decoder(z) # reconstructed x
    recon_loss = MSE(x, output)
    kl_loss = -0.5 * sum(1 - z_var - z_mean2 + log(z_var)) ! # KL loss
    loss = recon_loss + kl_loss
    loss_optimizer([encoder_weights, decoder_weights])
```

INFERENCE (i.e. generate on image)

TODO (for gen)